

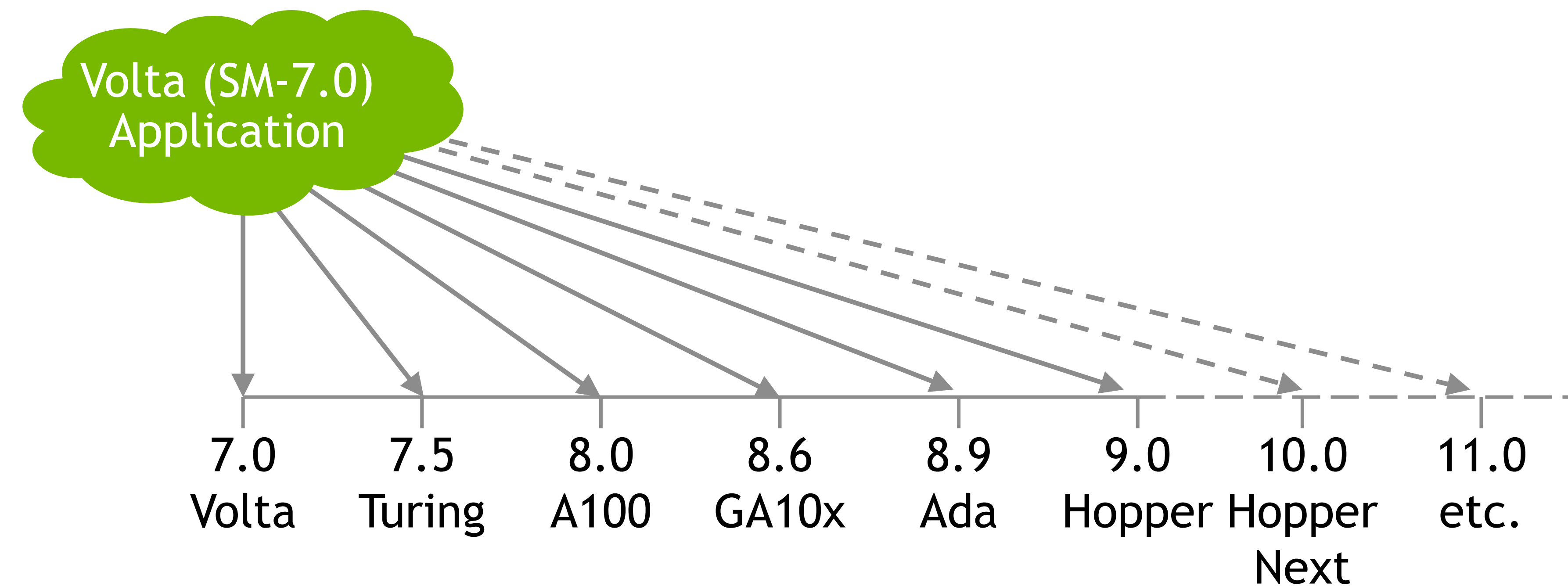


A41147 - Enabling Hopper-Specific Optimizations in CUDA Applications

Stephen Jones / Rob Armstrong – GTC – September 2022

“CUDA Everywhere”

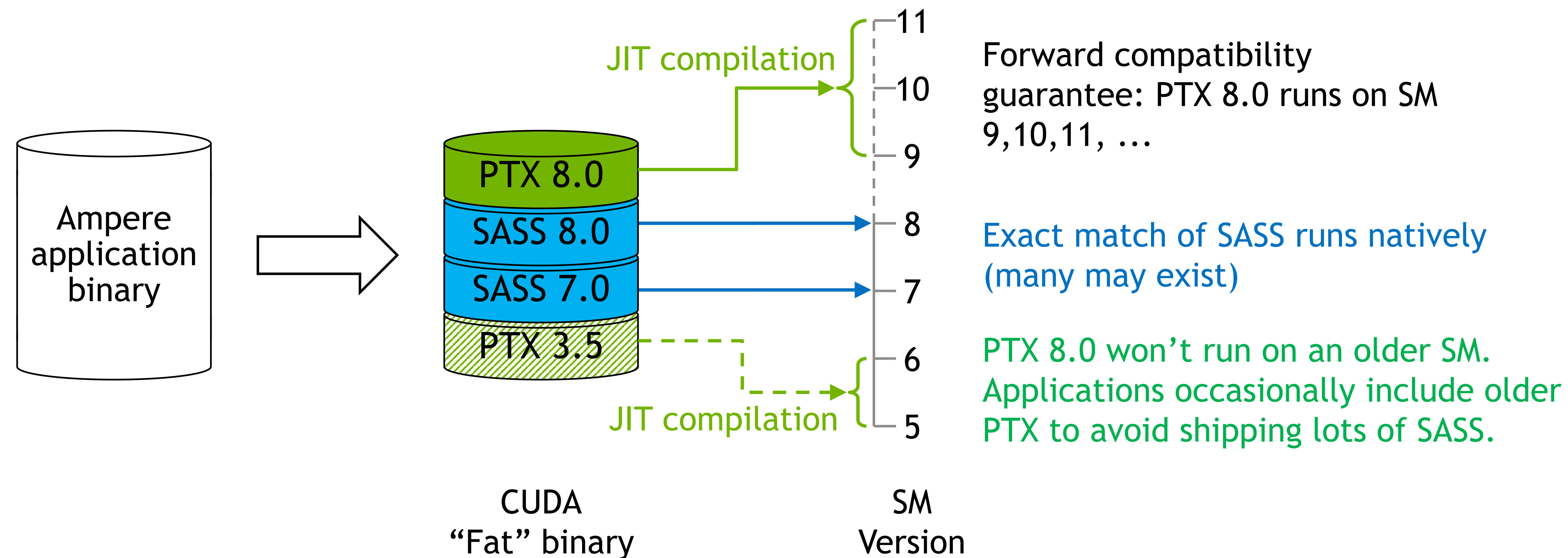
Code for one GPU runs on any GPU with a newer SM version



Volta applications “just work” on Turing/Ampere/Hopper
Datacenter libraries “just work” for Quadro, GeForce, etc.

Portability via PTX Just-In-Time Compilation

A CUDA application binary contains multiple types of code

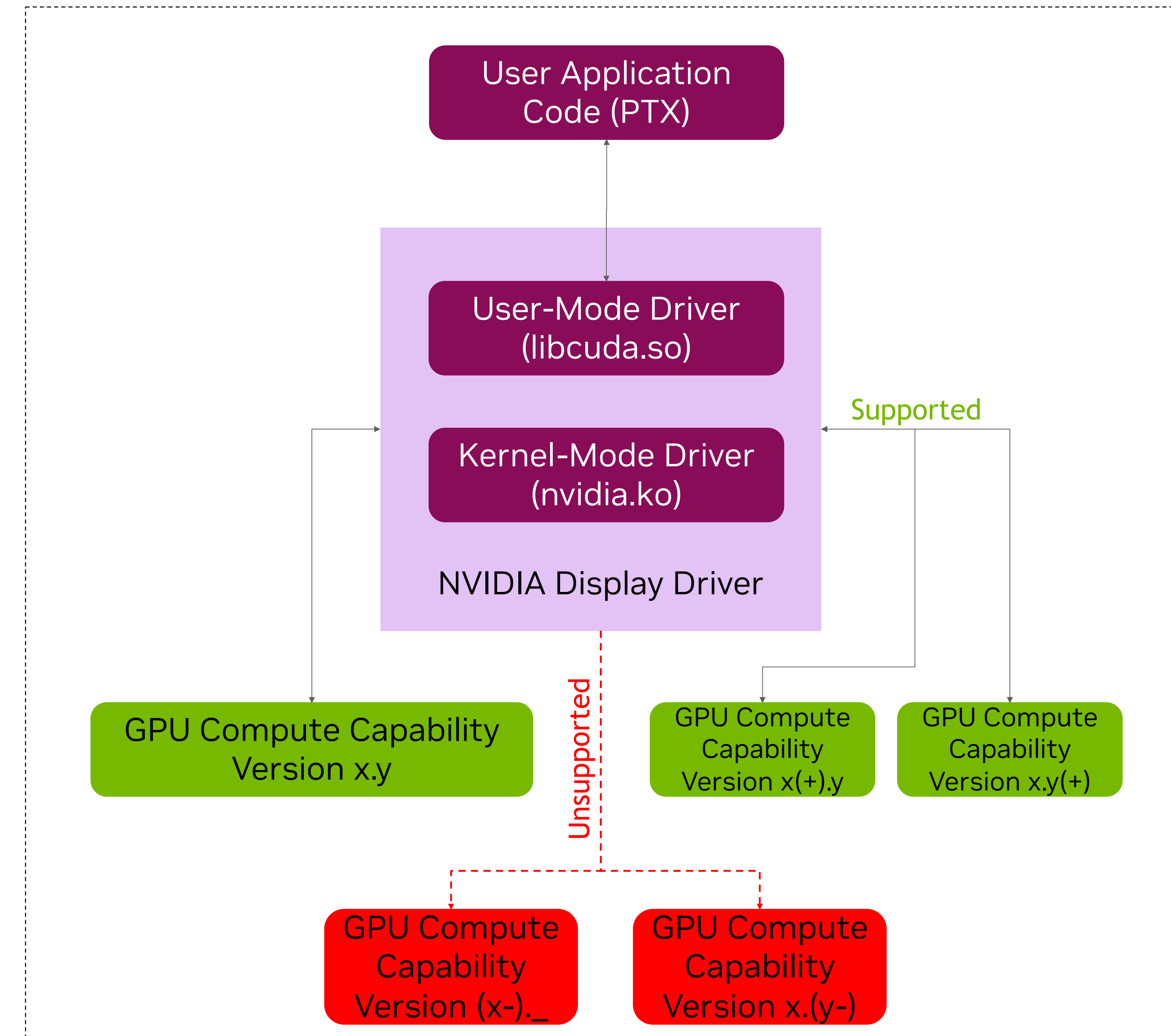


PTX is assembly code that is JIT compiled by CUDA when an application is run on a new GPU for which there is no SASS

Cuda compatibility

“PTX Compatibility” Hardware Considerations

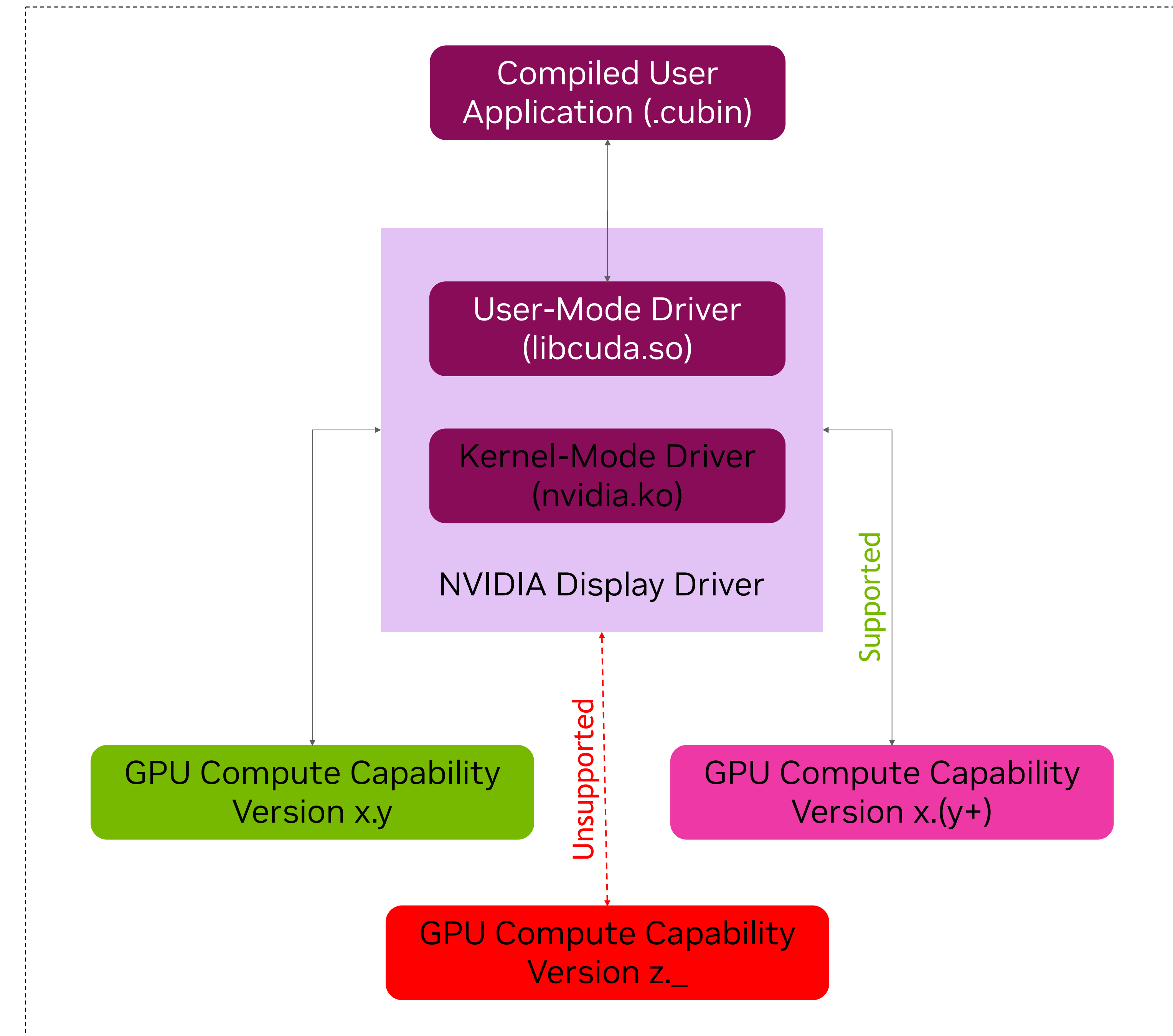
- ▶ **PTX Code** is compatible with future versions, both Major and Minor
- ▶ Supported PTX Migration:
 - ▶ CC 8.0 PTX runs on CC 8.6 (A100 PTX → A40)
 - ▶ CC 7.0 PTX runs on CC 8._ (V100 PTX → A100)
- ▶ Unsupported PTX Migration:
 - ▶ CC 8.6 PTX cannot run on CC 8.0 (A40 PTX → A100)
 - ▶ CC 8.0 PTX cannot run on CC 7.0 (A100 PTX → V100)



Cuda compatibility

“SM/Compute Compatibility” Hardware Considerations (Binaries)

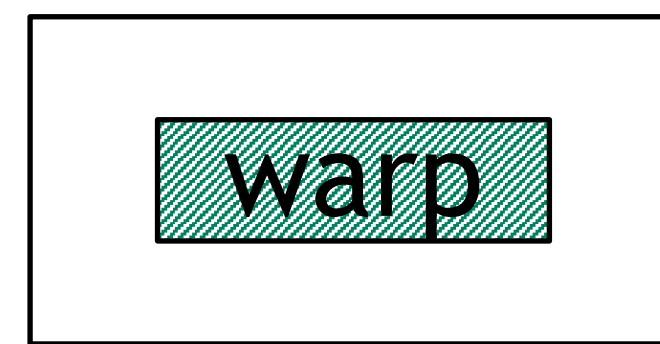
- ▶ While **binaries** may be backwards compatible with the driver, specific features unique to a Compute Capability of a GPU family may not be supported
- ▶ Backwards compatibility is guaranteed for only **previous minor-versions** compute capability
- ▶ Supported:
 - ▶ **CC 8.0** cubin runs on **CC 8.6** (A100 → A40)
- ▶ Unsupported:
 - ▶ **CC 8.6** cubin cannot run on **CC 8.0** (A40 → A100)
 - ▶ **CC 8.0** cubin cannot run on **CC 7.0** (A100 → V100)
 - ▶ **CC 7.0** cubin cannot run on **CC 8.0** (V100 → A100)



What Portability Looks Like

Generation-to-Generation Portability

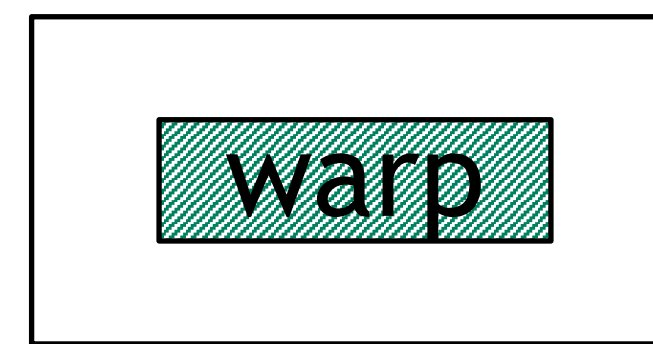
A100 Tensor Core



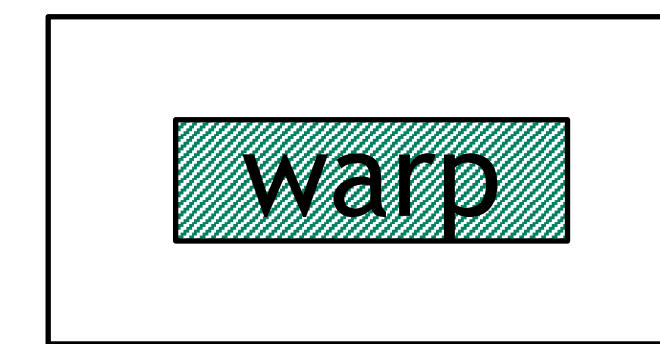
1 warp
(32 threads)



H100 Tensor Core



Future Tensor Core



Architecture-Specific Hardware Accelerators

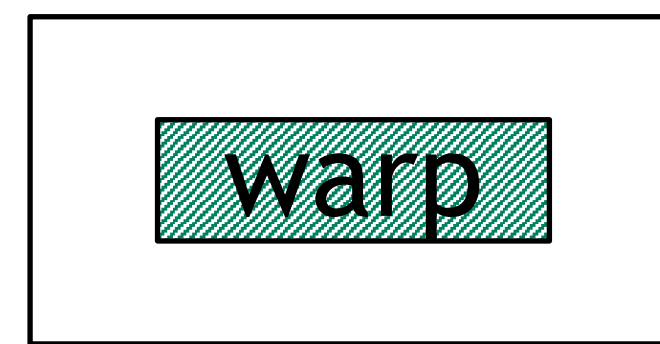
- ▶ With the introduction of the Hopper family we begin introducing architecture-specific hardware accelerators that are *not* JIT-compatible with future architectures
 - ▶ For instance, GMMMA on Hopper
- ▶ These features are *opt-in* to use in your own code, and are not compatible with future hardware
- ▶ This means:
 - ▶ Your code must include a compatible code path, if you want to JIT to other architectures
 - ▶ You may desire to QA both code paths
 - ▶ You may need to change your build process a bit to use these new capabilities

NEW: Architecture-Specific “Special Accelerated Features”

Ultra-high-performance features will exist on Hopper and future GPUs

Hopper Tensor Cores are unique to H100

A100 Tensor Core

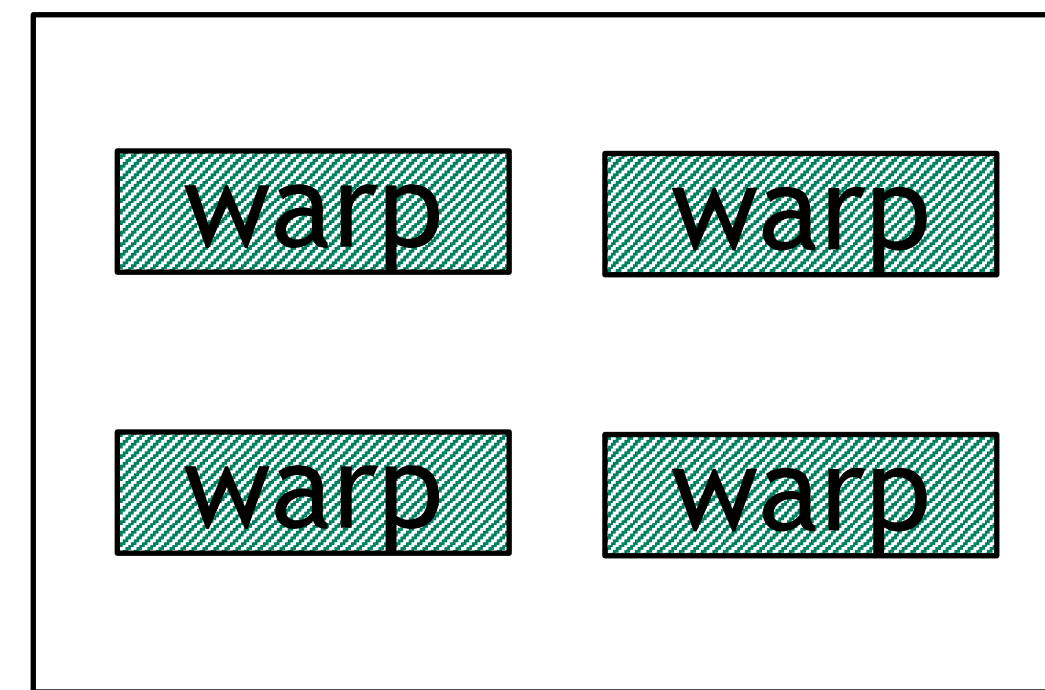


1 warp
(32 threads)

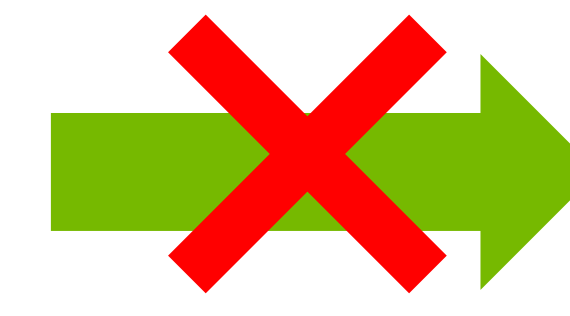


JIT cannot
create new
threads

H100 Tensor Core
with Opt-In Performance

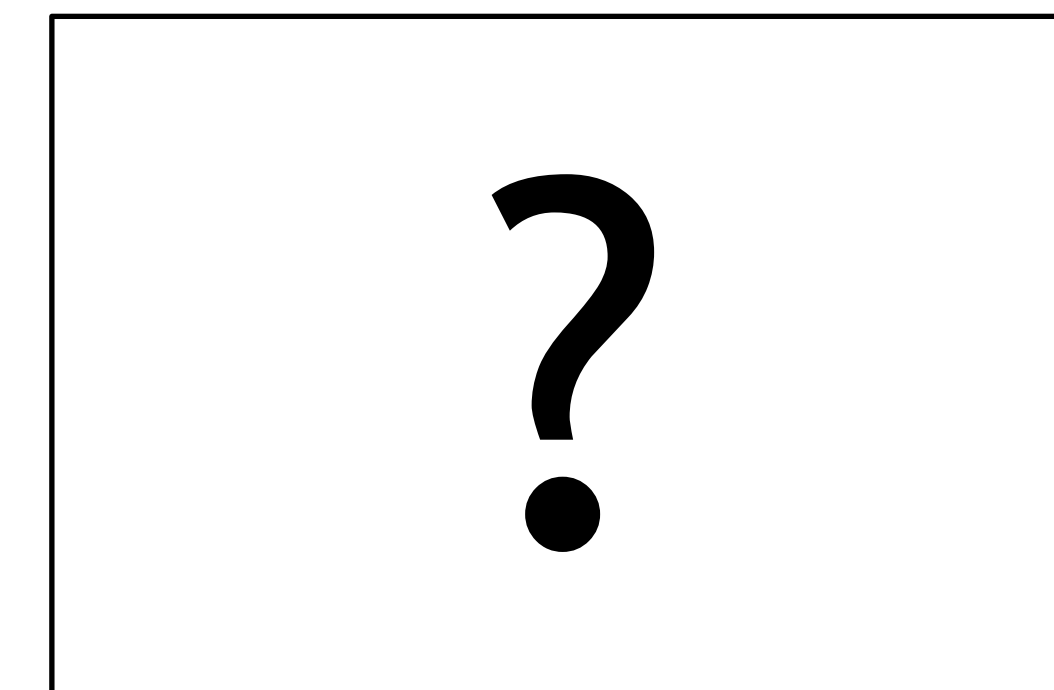


4 warp
(128 threads)



Cannot JIT
accelerated
features

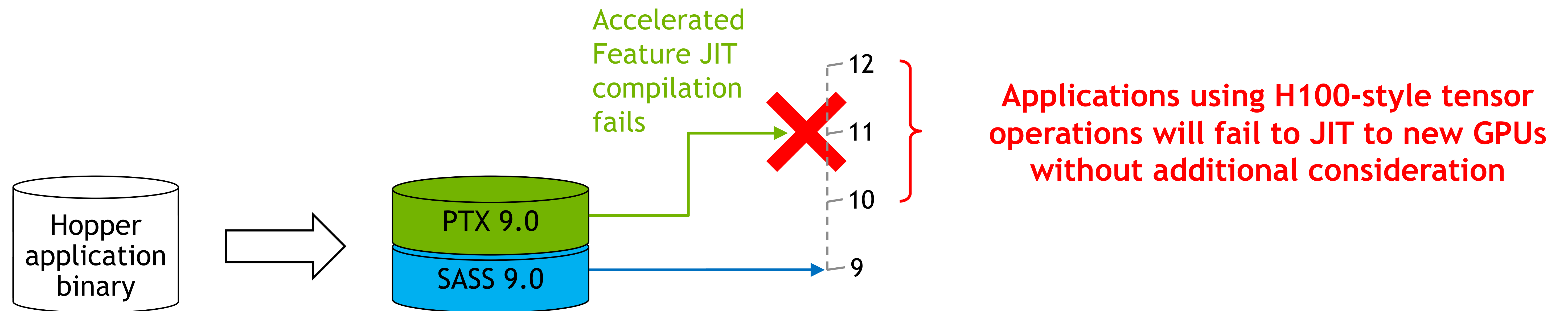
Future Tensor Core



New Design
(different to H100)

Why is this a concern?

For the first time, old CUDA applications may not run on new hardware

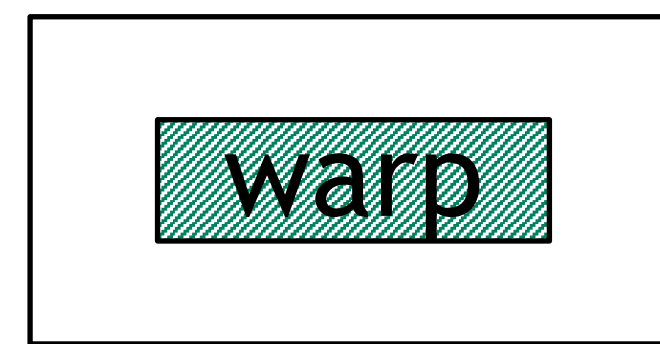


Hardware architecture innovation limits portability of special accelerated features

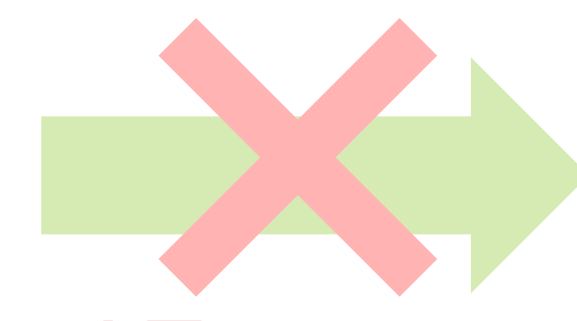
What are we doing about it?

Hardware **always** includes a forever-portable (but not peak performance) alternative

A100 Tensor Core

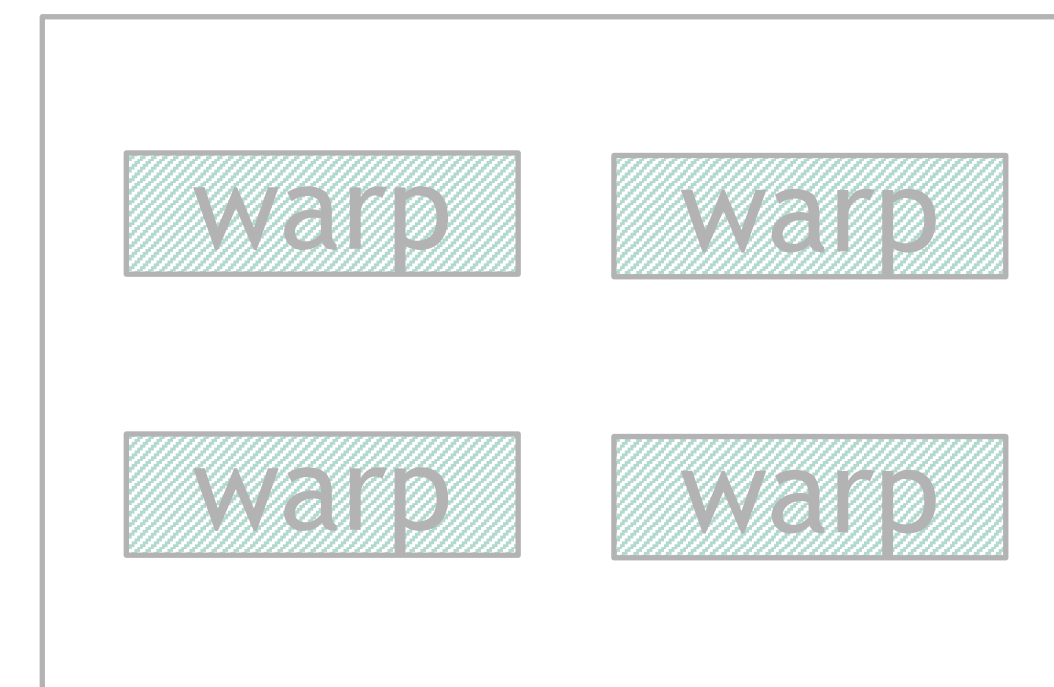


1 warp
(32 threads)

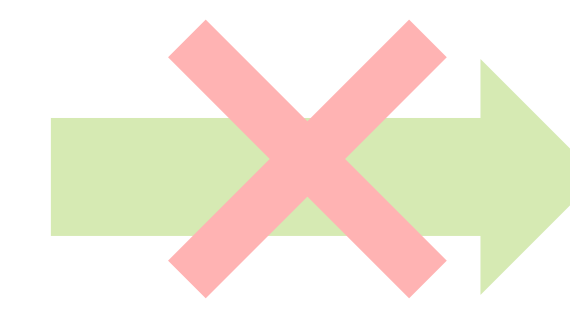


JIT cannot
create new
threads

H100 Tensor Core
with Opt-In Performance



4 warp
(128 threads)



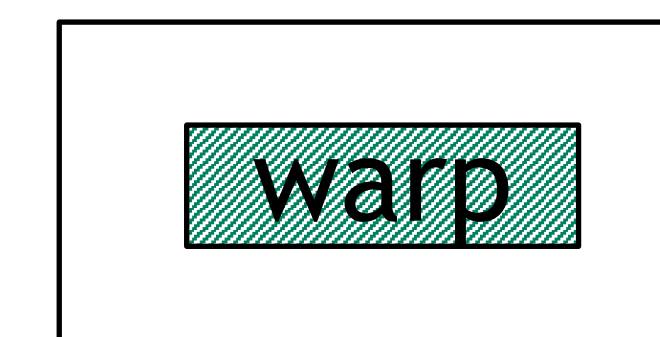
Cannot JIT
accelerated
features

Future Tensor Core



New Design
(different to H100)

Future
Tensor Core



“Standard” forever-portable Tensor Core will always JIT successfully to future architectures

What does a developer do about it?

MUST ALWAYS provide an alternative path for any Special Accelerated Feature

MMA_kernel.cu

```
__global__ void MMA kernel(...) {  
#if __CUDA_ARCH_HAS_FEATURE__(SM_90_ALL)  
// NOTE: Only works on H100  
...  
accelerated_mma();  
...  
#else  
// Fallback code for all other architectures  
...  
compatible_mma();  
...  
#endif  
}
```

Multi-Feature Compilation

New macro
__CUDA_ARCH_HAS_FEATURE__

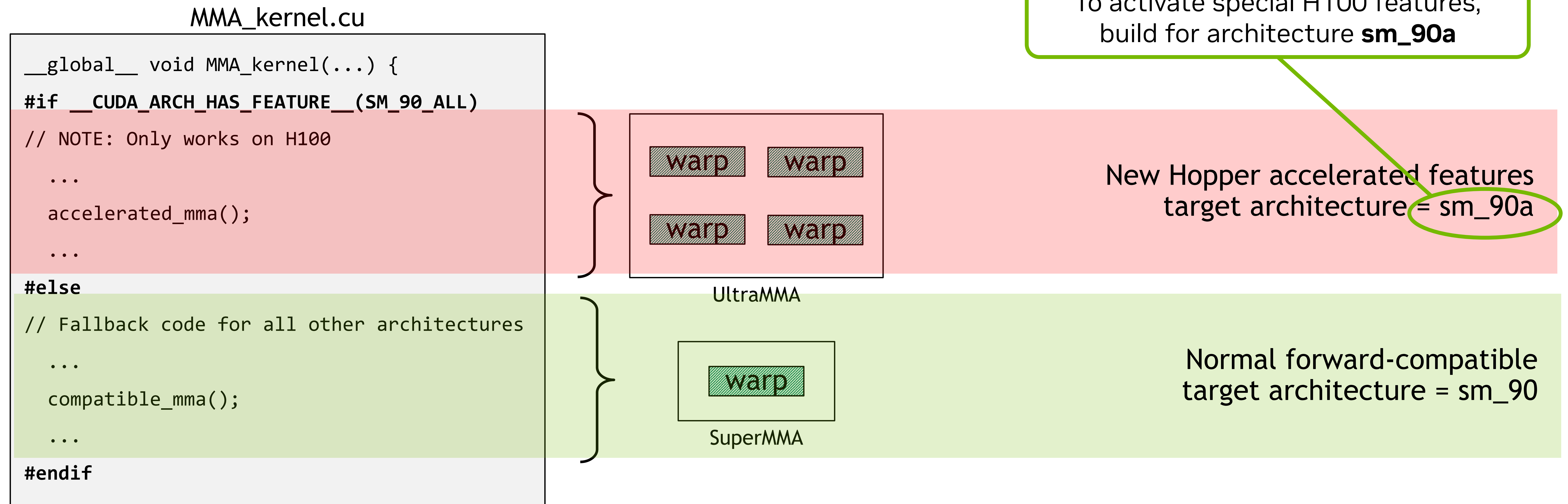
must protect all Hopper-specific code
(similar to using __CUDA_ARCH__)

NOTE

This is already
built-in to
NVIDIA libraries
such as
cuBLASDx or
CUTLASS

How does this work?

New target architecture “sm_90a” activates Hopper-specific features



Multi-Feature Compilation

using new macro `__CUDA_ARCH_HAS_FEATURE__`
(similar to using `__CUDA_ARCH__`)

Changes to how nvcc compiles code

Building for Hopper “sm_90” **DOES NOT** support special accelerated features

MMA_kernel.cu

```
__global__ void MMA_kernel(...) {  
#if __CUDA_ARCH_HAS_FEATURE__(SM_90_ALL)
```

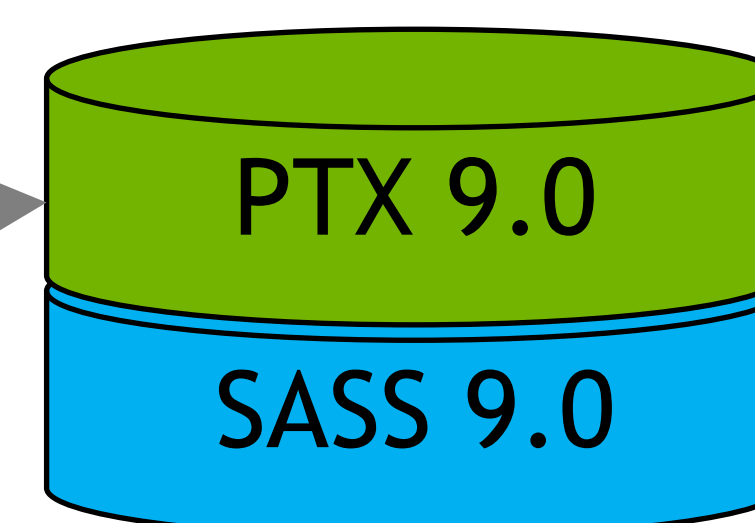
DISABLED

```
#else  
// Fallback code for all other architectures  
...  
compatible_mma();  
...  
#endif
```

```
$ nvcc -arch=sm_90 MMA_kernel.cu
```

Special features not available
for arch=sm_90

JIT compilation
succeeds



Binary contains fully-portable PTX-9.0

12
11
10
9

Multi-Feature Compilation

using new macro `__CUDA_ARCH_HAS_FEATURE__`
(similar to using `__CUDA_ARCH__`)

Opt-in to sm_90a architecture to enable accelerated features

To ensure JITtable binary, compiler includes **BOTH** accelerated & portable PTX

MMA_kernel.cu

```
__global__ void MMA_kernel(...) {  
    #if __CUDA_ARCH_HAS_FEATURE__(SM_90_ALL)  
    // NOTE: Only works on H100  
    ...  
    accelerated_mma();  
    ...  
    #else  
    // Fallback code for all other architectures  
    ...  
    compatible_mma();  
    ...  
    #endif  
}
```

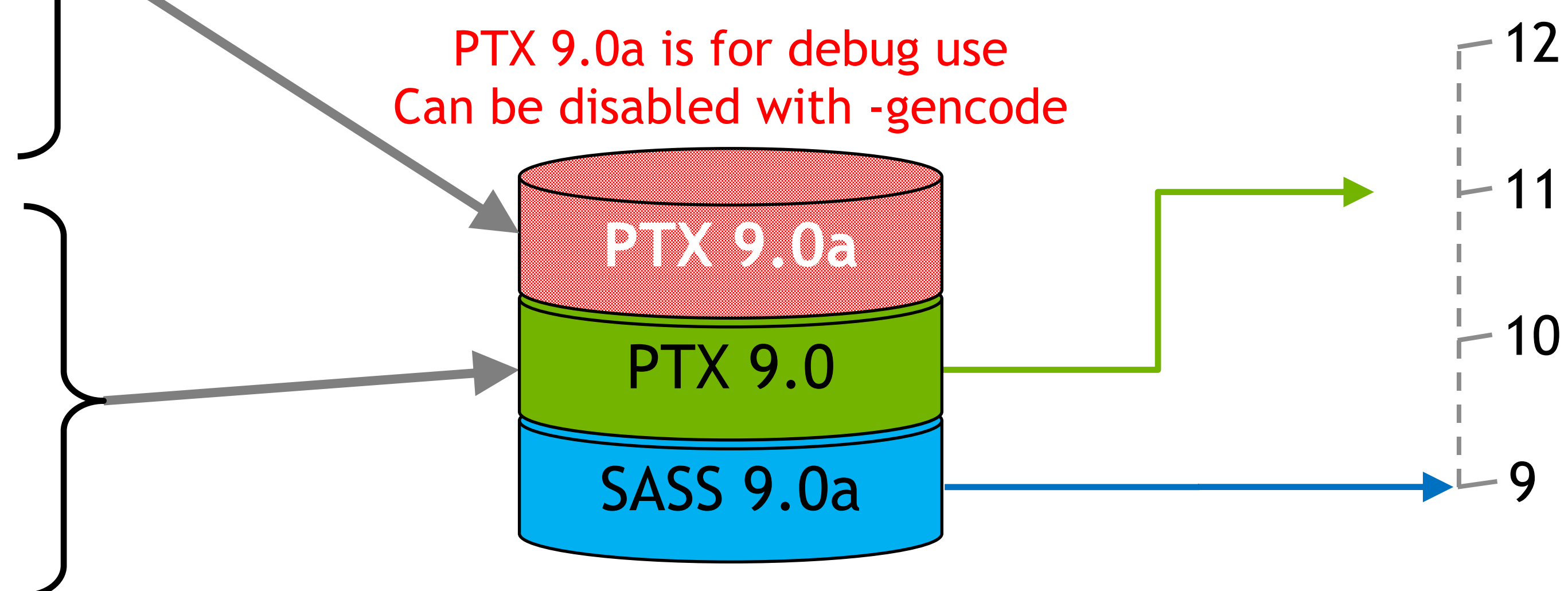
Multi-Feature Compilation

using new macro `__CUDA_ARCH_HAS_FEATURE__`
(similar to using `__CUDA_ARCH__`)

```
$ nvcc -arch=sm_90a MMA_kernel.cu
```

Compiler builds **TWICE**, once for sm_90a and again for sm_90, to ensure JITtable PTX in binary

PTX 9.0a is for debug use
Can be disabled with -gencode



Binary contains fully-portable PTX-9.0
AND Hopper-specific PTX-9.0a

Opt-in to sm_90a architecture to enable accelerated features

To ensure JITtable binary, compiler includes **BOTH** accelerated & portable PTX

MMA_kernel.cu

```
__global__ void MMA_kernel(...) {  
    #if __CUDA_ARCH_HAS_FEATURE__(SM_90_ALL)  
        // NOTE: Only works on H100  
        ...  
        accelerated_mma();  
        ...  
    #else  
        // Fallback code for all other architectures  
        ...  
        compatible_mma();  
        ...  
    #endif  
}
```

```
$ nvcc -arch=sm_90a MMA_kernel.cu
```

```
.target sm_90a
```

```
...
```

```
gmma
```

```
...
```

If accelerated MMA is found
under target "sm_90" then
PTX compiler will give an
error

```
.target sm_90
```

```
...
```

```
hmma
```

```
...
```

Output is two PTX files with
different target architectures

Multi-Feature Compilation

using new macro `__CUDA_ARCH_HAS_FEATURE__`
(similar to using `__CUDA_ARCH__`)

Developer rules for using Special Accelerated Features

Easy for developers, but extra care for QA

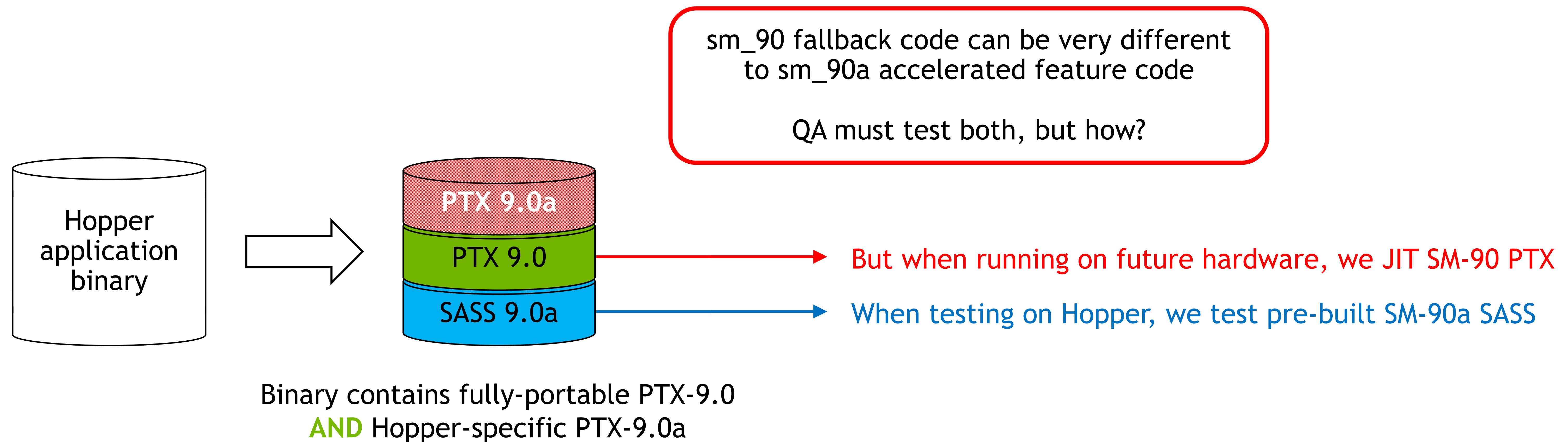
Basic rule: Opt-in to high performance Tensor Cores by using CUTLASS and just set the compiler flag

1. By default, Special Accelerated Features are disabled (e.g. Hopper compilation `-arch=sm_90`)
2. Opt-in to access architecture-specific high performance features via `nvcc -arch=sm_90a` – you will also need to do this if you use CUDA C++ libraries such as CUTLASS
3. **Must** include a non-special-feature path for future hardware portability (`__CUDA_ARCH_HAS_FEATURE__`)
4. Do **not** always just build with `-arch=sm_90a` - this increases build time, file size and QA time, so use it only when you need it
5. Application testing must now test both paths (see next slides)

The real difference: QA

Must test BOTH codepaths, for host and device code

Question 1: How do we test the fallback code path?



The real difference: QA

Must test BOTH codepaths, for host and device code

Question 2: How do we test host code which must launch with different resources?

host_code.cu

```
void main() {
    cudaDeviceProp prop;
    cudaDeviceGetProperties(&prop, 0);

    // For SM-9.0, UltraMMA is 128-threads wide
    // so launch with large block and large SMEM
    if(prop.major == 9) {
        MMA_Kernel <<< griddim, 128, 96*1024 >>>( ... );
    }
    // For other architectures, SuperMMA is 32-threads
    // wide so launch with smaller block size & SMEM
    else {
        MMA_Kernel <<< griddim, 64, 64*1024 >>>( ... );
    }
    cudaDeviceSynchronize();
}
```

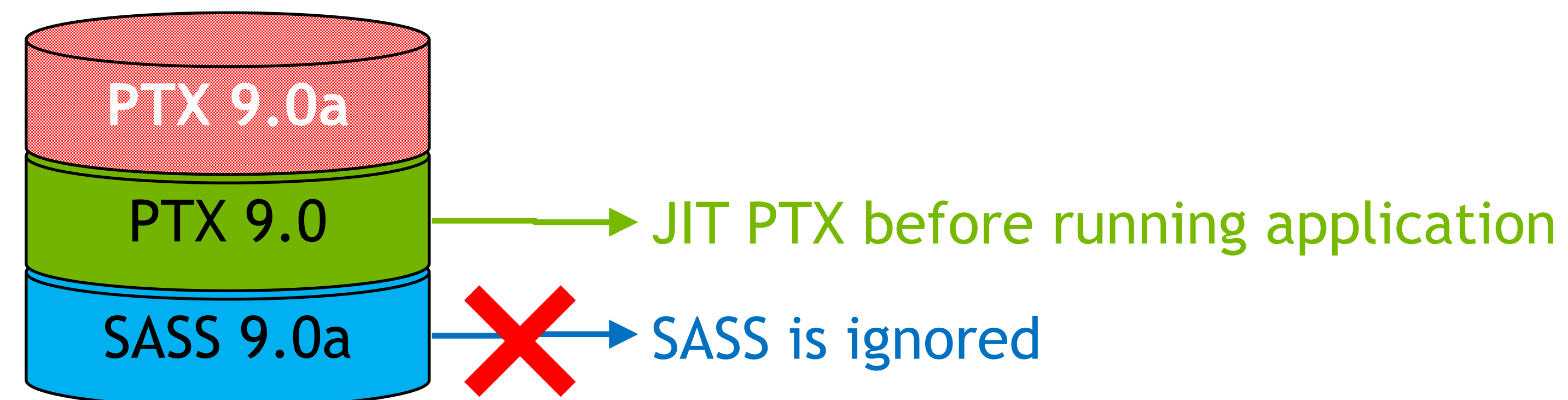
QA needs to activate **both** Hopper launch path **and** other-architecture launch path because code paths are different

New environment variable for QA to test JIT path

General Rule: Always test portable code path on same architecture as accelerated features
(otherwise could end up with PTX version mismatches)

CUDA_TEST_UNKNOWN_FUTURE_SM_MAJOR=1

Forces JIT even if SASS is present



Spoofs SM major version to exercise SM-specific host code paths
Also spoofs reported resources (SMEM, etc) to be portable sizes

```
void main() {  
    cudaDeviceProp prop;  
    cudaDeviceGetProperties(&prop, 0);  
  
    if(prop.major == 9) {  
        ... // CUDA reports not SM-9.0  
    } else {  
        ... // Spoofing forces this path  
    }  
}
```


QA rules for using Special Accelerated Features

QA effectively needs to test another GPU architecture – “Unknown Future Architecture”

1. If code opts-in to Accelerated Features, QA **must** test fallback path as well
2. Only applies to CUDA C++ device code – apps using only NVIDIA host libraries do not require extra testing
3. It may not always be clear when you are using Accelerated Features – you may link a library which uses it internally (such as CUTLASS)
4. Identify applications by detecting opt-in via compiler `-arch=sm_90a` flag
5. Not good enough just to test sm_90a kernels: host code may be different as well so retest **everything**
6. Best practice: test using `CUDA_TEST_UNKNOWN_FUTURE_SM_MAJOR=1` (will also force JIT)

Summary

- Hopper architecture brings high-performance MMA capabilities, but that hardware must be driven in a way that can't be JIT-compiled from previous architectures
- Future GPU architectures may include similar “special accelerated features”
- As a developer:
 - Add accelerated and portable code paths
 - NVIDIA-provided libraries will include these by default
- As a tester:
 - Ensure both accelerated and fallback code paths are tested for both host and device
 - Ensure JIT compilation of fallback code path is tested

